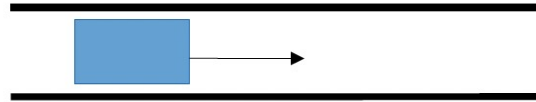


The problem

A two way east-west road contains a narrow bridge with only one lane. An eastbound (or westbound) car can pass over the bridge only if there is no oncoming car on the bridge. Traffic may only cross the bridge in one direction at a time, and if there are ever more than 3 vehicles on the bridge at one time, it will collapse under their weight. In this system, each car is represented by one thread, which executes the procedure `OneVehicle` when it arrives at the bridge.



```
OneVehicle(Direction direc) {  
    ArriveBridge(direc);  
    CrossBridge(direc);  
    ExitBridge(direc);  
}
```

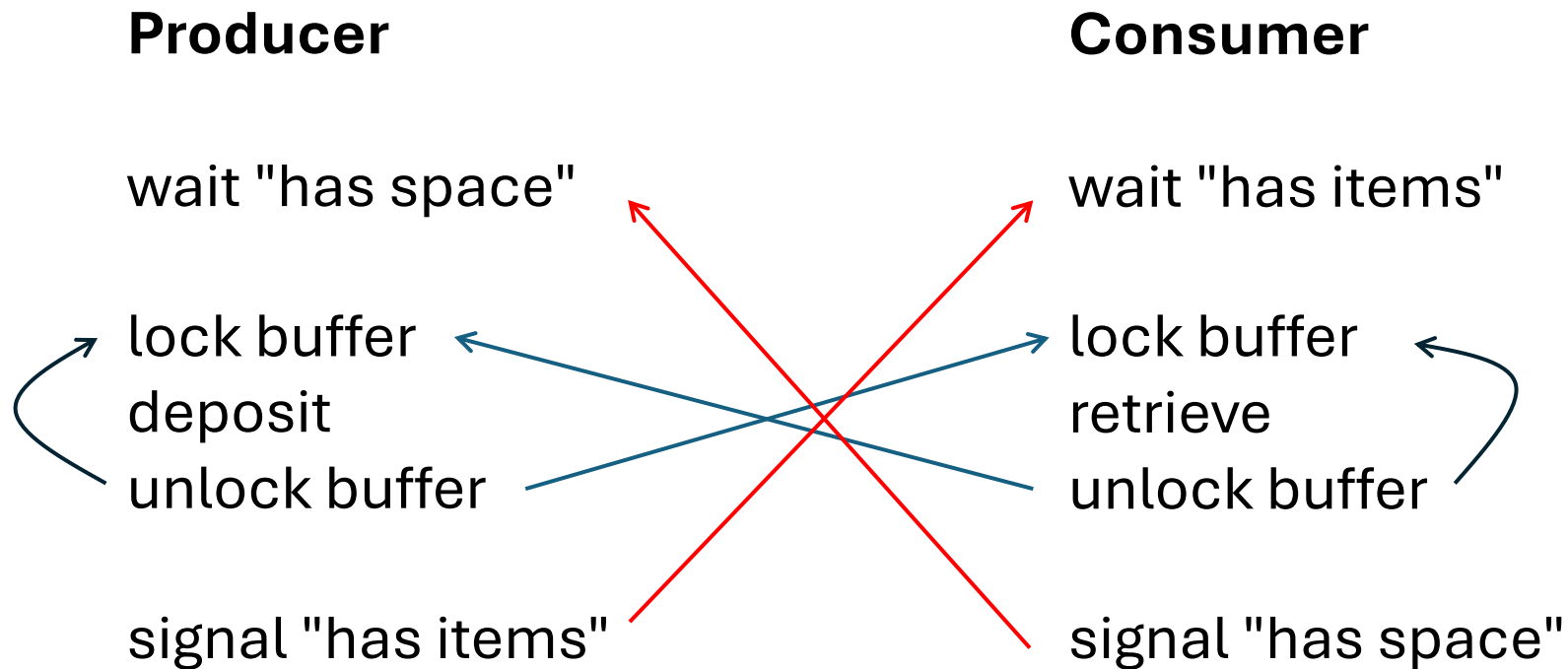
`direc` gives the direction in which the vehicle will cross the bridge

Write the procedures `ArriveBridge` and `ExitBridge`.

`ArriveBridge` must not return until it safe for the car to cross the bridge in the given direction (it must guarantee that there will be no head-on collisions or bridge collapses).

`ExitBridge` is called to indicate that the caller has finished crossing the bridge; `ExitBridge` should take steps to let additional cars cross the bridge.

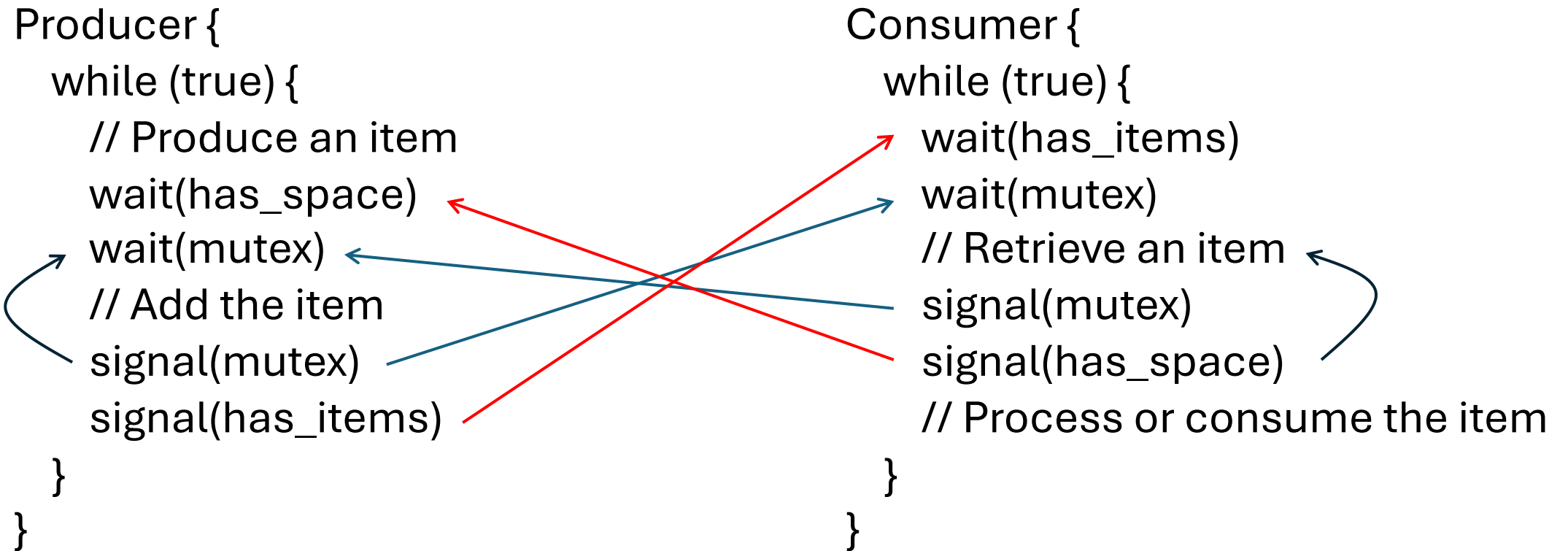
We want to use conditional variables but, before we discuss them, let's go back to producer-consumer implemented with semaphores



binary semaphore mutex = 1

general semaphore has_items = 0

general semaphore has_space = N



What are the limitations?

- It works only for counter-based synchronization, but a single semaphore count cannot directly represent a Boolean condition like `cars < 3 AND (cars == 0 OR current_direction == my_direction)`
- The condition is not explicit
 - `wait(has_space)` suggests the meaning through the name, but it does not explicitly check `while (count == max_size)`
- `signal(has_space)` increments the semaphore even if nobody is currently waiting. Also, Therefore, one accidental extra signal can create a false permit

Can we use locks?

- Lock: token that can be held by 0 or 1 thread at a time
- `acquire`: get the lock if the lock is free, wait otherwise
- `release`: mark the lock as free (assumption: only how owns the lock calls `release`)

Let's build prod/cons bounded buffer with locks (starting from scratch) – attempt 1

```
shared count = 0  
shared max_size = N
```

```
Producer (repeat forever) {  
01 // Produce an item  
02 while (count == max_size) {}  
03 // Add the item  
04 count++;  
}
```

```
Consumer (repeat forever) {  
01 while (count == 0) {}  
02 // Retrieve an item  
03 count--;  
04 // Consume the item  
}
```

Let's build prod/cons bounded buffer with locks (starting from scratch) – attempt 1

```
shared count = 0
shared max_size = N
```

```
Producer (repeat forever) {
01 // Produce an item
02 while (count == max_size) {}
03 // Add the item
04 count++;
}
```

```
Consumer (repeat forever) {
01 while (count == 0) {}
02 // Retrieve an item
03 count--;
04 // Consume the item
}
```

2 Producers, P1 P2
N=20



```
... // here count=19
P1:02
P1:03
Context switch (P1 → P2)
P2: 02
P2: 03
P2: 04 (count is now 20)
Context switch (P2 → P1)
P1: 04 (count is now 21)
```

Let's build prod/cons bounded buffer with locks (starting from scratch) – attempt 2

```
shared count = 0
shared max_size = N
shared lock l
```

Producer (repeat forever) {	Consumer (repeat forever) {
01 // Produce an item	01 acquire(l);
02 acquire(l);	02 while (count == 0) {}
03 while (count == max_size) {}	03 // Retrieve an item
04 // Add the item	04 count--;
05 count++;	05 release(l);
06 release(l);	06 // Consume the item
}	}

Let's build prod/cons bounded buffer with locks (starting from scratch) – attempt 2

```
shared count = 0
shared max_size = N
shared lock l
```

```
Producer (repeat forever) {
01 // Produce an item
02 acquire(l);
03 while (count == max_size) {}
04 // Add the item
05 count++;
06 release(l);
}
```

```
Consumer (repeat forever) {
01 acquire(l);
02 while (count == 0) {}
03 // Retrieve an item
04 count--;
05 release(l);
06 // Consume the item
}
```



```
1 Producer, P1
1 Consumer, C1
N=20
Initially empty

C1: 01 // Game over
```

Let's build prod/cons bounded buffer with locks – attempt 3

shared count = 0
shared max_size = N
shared lock l

Producer (repeat forever) {

01 // Produce an item

02 acquire(l);

03 while (count == max_size) {

04 release(l);

05 acquire(l);

06 }

07 // Add the item

08 count++;

09 release(l);

}

Consumer (repeat forever) {

01 acquire(l);

02 while (count == 0) {

03 release(l);

04 acquire(l);

05 }

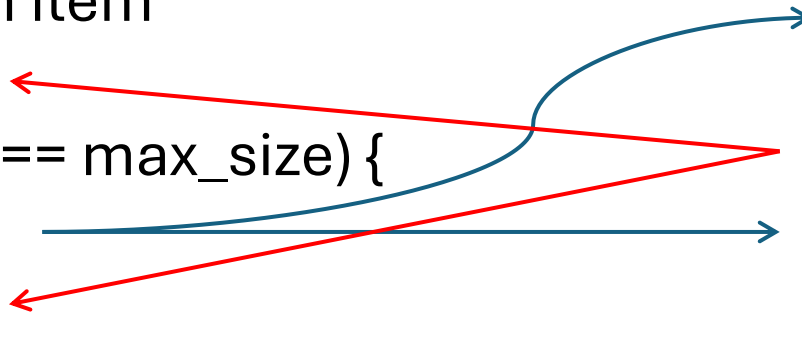
06 // Retrieve an item

07 count--;

08 release(l);

09 // Consume the item

}



Let's build prod/cons bounded buffer with locks – attempt 3

shared count = 0
shared max_size = N
shared lock l

- It's not guaranteed that the producer(s) / consumer(s) will make progress
- What's good: that both release the lock
- What's bad: that both wake up and try to get the lock independently of whether they can actually do their job

Producer (repeat forever) {

01 // Produce an item

02 acquire(l);

03 while (count == max_size) {

04 release(l);

05 acquire(l);

06 }

07 // Add the item

08 count++;

09 release(l);

}

Consumer (repeat forever) {

01 acquire(l);

02 while (count == 0) {

03 release(l);

04 acquire(l);

05 }

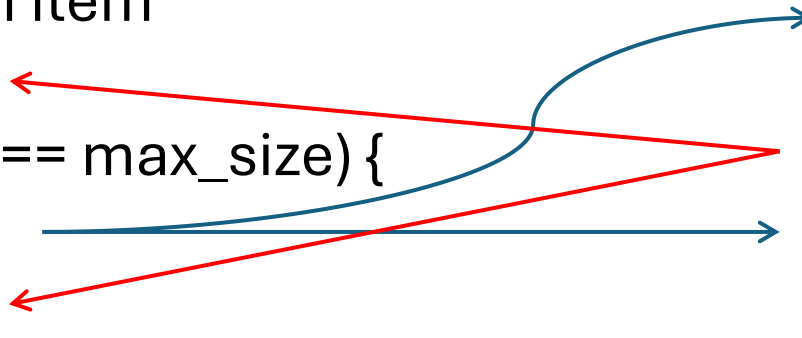
06 // Retrieve an item

07 count--;

08 release(l);

09 // Consume the item

}



Condition variable: an object that you can use to wait for a condition to become true, associated with a lock.

`cond_wait(condition, lock):`

- Release the lock.
- The thread goes to sleep / enters the waiting queue.
- Upon waking up, it reacquires the lock.

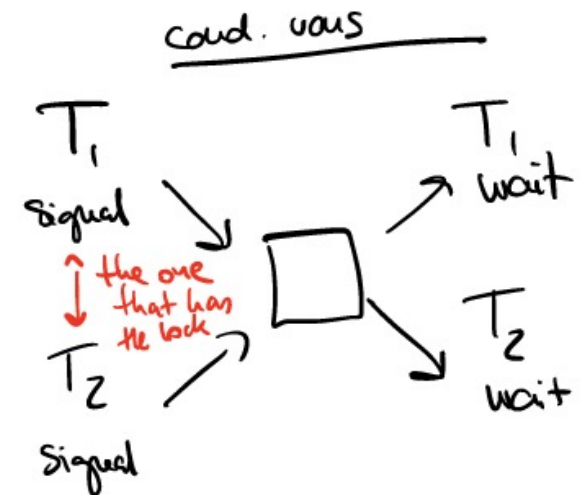
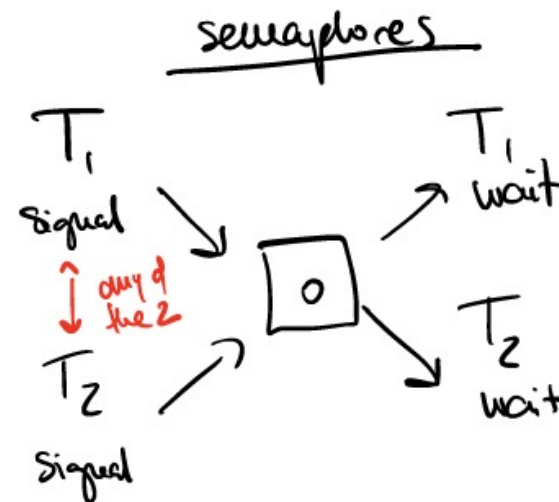
`cond_signal(condition, lock):`

- If a thread is waiting on the condition, wake up one and give it the lock.

`cond_broadcast(condition, lock):`

- Same as signal, except all waiting threads wake up.

- Whoever calls these methods needs to hold the lock.
- Fine-grained control over execution order.

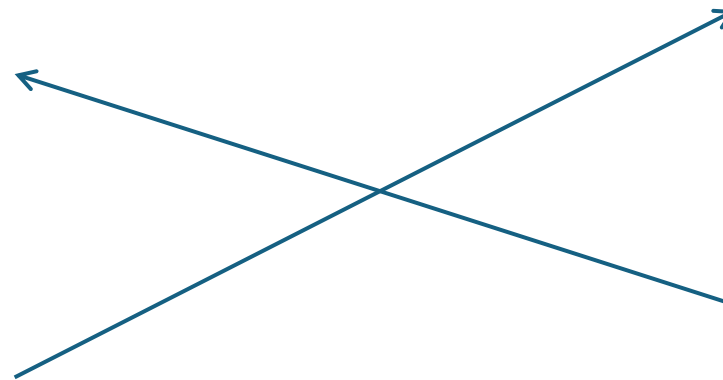


Let's build prod/cons bounded buffer with locks (starting from scratch) – attempt 4

shared count = 0
shared max_size = N
shared lock l
condition has_space
condition has_items

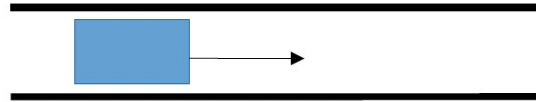
```
Producer (repeat forever) {  
01 // Produce an item  
02 acquire(l);  
03 while (count == max_size) {  
04   cond_wait(has_space, l);  
05 }  
06 // Add the item  
07 count++;  
08 cond_signal(has_items, l);  
09 release(l);  
}
```

```
Consumer (repeat forever) {  
01 acquire(l);  
02 while (count == 0) {  
03   cond_wait(has_items, l);  
04 }  
05 // Retrieve an item  
06 count--;  
07 cond_signal(has_space, l);  
08 release(l);  
09 // Consume the item  
}
```



The problem

A two way east-west road contains a narrow bridge with only one lane. An eastbound (or westbound) car can pass over the bridge only if there is no oncoming car on the bridge. Traffic may only cross the bridge in one direction at a time, and if there are ever more than 3 vehicles on the bridge at one time, it will collapse under their weight. In this system, each car is represented by one thread, which executes the procedure `OneVehicle` when it arrives at the bridge.



```
OneVehicle(Direction direc) {  
    ArriveBridge(direc);  
    CrossBridge(direc);  
    ExitBridge(direc);  
}
```

`direc` gives the direction in which the vehicle will cross the bridge

Write the procedures `ArriveBridge` and `ExitBridge`.

`ArriveBridge` must not return until it safe for the car to cross the bridge in the given direction (it must guarantee that there will be no head-on collisions or bridge collapses).

`ExitBridge` is called to indicate that the caller has finished crossing the bridge; `ExitBridge` should take steps to let additional cars cross the bridge.

Constraints

- $0 \leq \text{cars} \leq 3$
- All cars in the same direction
- Bridge empty ($\text{cars} == 0$) AND car wants in? OK
- Bridge NOT(empty OR full) AND car in the same direction? OK
- Synchronization shared variables:
 - cars
 - direction

shared variable cars = 0

shared Direction D = 0

lock bridge

waitingToGo[2]

ArriveBridge(Direction d) {

 lock(bridge);

 while (cars == 3 || (cars > 0 && D != d)) {

 waitingToGo[d]++;

 cond_wait(waitingToGo[d], bridge);

 waitingToGo[d]--;

 }

 cars++;

 D = d;

 release(bridge);

}

ExitBridge(Direction d) {

 lock(bridge);

 cars--;

 if (waitingToGo[d] > 0) {

 cond_signal(waitingToGo[d], bridge);

 } else if (cars == 0) {

 cond_broadcast(waitingToGo[1 - d],

bridge);

 }

 release(bridge);

}